UADE

Trabajo Práctico Integrador — Programación 1 / Algoritmo y Estructura de Datos 1

DOCUMENTO EXPLICATIVO Y TÉCNICO — FASE 2: AMPLIACIÓN Y MEJORA DEL SISTEMA

Sistema: Room Master (Gestión Hotelera) **Docente Titular:** Lic. Gustavo E. Escandell

Ciclo Académico: 2° Cuatrimestre — Año 2026 **Fecha de Entrega Oficial:** 05/11/2026

Equipo de Trabajo: Menendez Mojana, Tiara (Referente) • Ramos, Lucas Sebastián • Iglesias, Luca • Rueda, Leandro Nicolás

1. Descripción de las Mejoras Respecto a la Primera Parte (Ítem a)

En esta segunda entrega, el proyecto evolucionó desde una versión inicial basada en listas genéricas hacia una arquitectura modular robusta y desacoplada, dividida en 7 archivos con responsabilidades claras:

- **Modularización desacoplada:** División funcional en `hotel.py` (matriz y renderizado), `tarifas.py` (esquemas de precios), `huespedes.py` (operaciones con clientes), `reportes.py` (análisis funcional), `validaciones.py` (expresiones regulares), `archivos.py` (persistencia) y `main.py` (coordinador).
- **Reubicación de Huéspedes (Swap):** Capacidad de trasladar huéspedes a habitaciones libres actualizando atómicamente la matriz y los registros en memoria.
- **Ciclo Sanitario de Habitaciones:** Incorporación del estado Sucio (`[S]`) tras el Check-out, garantizando que ninguna habitación liberada vuelva a Libre (`[L]`) sin un proceso explícito de restablecimiento y limpieza.
- **Búsquedas funcionales avanzadas:** Búsqueda secuencial por DNI, búsqueda parcial insensible a mayúsculas por nombre con `filter` y `lambda`, y búsqueda por habitación.
- **Esquema tarifario segmentado:** Configuración de recargo porcentual dinámico para pisos de Suites/Lujo y liquidación con bonificación del 10% mediante expresión `lambda` para estadías superiores a 7 noches.
- **Control universal de cancelación:** Soporte de interrupción con `-1` en todas las entradas de usuario, evitando bloqueos por errores de digitación.

2. Estructuras de Datos y Justificación Técnica (Ítem b)

2.1. Conjuntos (`set`)

Se utilizan en dos módulos críticos:

1. **Control de duplicados (`huespedes.py`):** En la función `validar_dni`, los DNIs registrados se almacenan en un conjunto. Dado que los conjuntos en Python se implementan sobre **tablas hash**, la verificación de existencia (`dni in dnis`) se ejecuta en complejidad temporal promedio de **O(1)** (tiempo constante), superando a la búsqueda lineal O(n) de una lista tradicional.
2. **Álgebra de disponibilidad (`reportes.py`):** En `reporte_habitaciones_libres`, se calculan las habitaciones desocupadas aplicando la operación matemática de **diferencia de conjuntos**:

   ```
   libres = todas_las_habitaciones - habitaciones_ocupadas
   ```

   Esto elimina bucles anidados manuales y permite verificar consistencia mediante la intersección (`todas & ocupadas`).

2.2. Tuplas (`tuple`)

Se emplean para representar la clave de identidad de los huéspedes:

```
huesped["identidad"] = (dni, nombre) # Tupla inmutable de 2 elementos
```

Justificación: Las tuplas garantizan **inmutabilidad** e integridad referencial. El DNI y nombre constituyen datos registrales legales del huésped que bajo ninguna circunstancia deben ser alterados accidentalmente durante operaciones de Check-in, Check-out o Swap de habitaciones. Además, se emplean en retornos múltiples de funciones (ej. coordenadas `piso, habitacion`).

2.3. Diccionarios (`dict`)

Cada huésped se representa como un diccionario con claves semánticas descriptivas:

```
datos_huesped = {
    "identidad": (dni, nombre),              # Tupla inmutable
    "dias": int(dias),                       # Entero positivo
    "ubicacion": [piso, habitacion, num_hab]  # Lista mutable (permite Swap)
}
```

Justificación: Permite acceso directo por clave en O(1) (ej: `huesped["dias"]`), eliminando la dependencia de índices posicionales rígidos que producían errores al agregar o mover campos en las listas de la primera entrega.

3. Persistencia de Datos (Ítem c)

En estricto cumplimiento de la consigna que prohíbe el uso de bases de datos, el sistema implementa un modelo de **triple persistencia desacoplada**:

Tipo de Archivo	Nombre del Archivo	Modo Apertura	Utilidad Concreta en el Sistema
JSON	`hotel_data.json`	`'r'` / `'w'`	**Persistencia del estado estructural:** Guarda las dimensiones edilicias, las tarifas por piso y la lista completa de diccionarios de huéspedes activos para reanudar el programa sin pérdida de datos.
Texto Plano	`operaciones.txt`	`'a'` (append)	**Bitácora transaccional continua:** Registra cronológicamente cada comprobante de Check-out en tiempo real sin tener que sobrescribir ni leer todo el archivo previo. Sirve de auditoría de caja.
CSV	`reporte_facturacion.csv`	`'w'` (escritura)	**Exportación gerencial:** Estructura el historial en filas y columnas estándar mediante `csv.writer`, permitiendo abrir la liquidación en Microsoft Excel o Google Sheets para balances contables.

> **Flujo Operativo de Recuperación:** Al ejecutar el sistema, `archivos.py` busca automáticamente `hotel_data.json` mediante un gestor de contexto `with open(...)`. Si existe, reconstruye el hotel en memoria; si no existe, captura la excepción `FileNotFoundError` e inicia la configuración guiada inicial sin detener el programa.

4. Manejo de Excepciones (Ítem d)

Se incorporó control de excepciones con bloques `try / except / else / finally` en situaciones reales de riesgo en tiempo de ejecución, evitando su uso indiscriminado:

- **`FileNotFoundError`:** Manejo de la ausencia del archivo de datos en la primera ejecución del sistema.
- **`PermissionError`:** Captura de bloqueos de permisos de lectura/escritura impuestos por el sistema operativo en archivos de persistencia.
- **`ValueError`:** Prevención de caídas ante intentos de parseo de datos no numéricos o estructuras JSON con formato dañado.
- **`IndexError`:** Validación de límites dimensionales en el acceso a celdas de la matriz del hotel.

- **Cláusula `finally`:** Garantía de cierre forzado de descriptores de archivos (`archivo.close()`) en caso de contingencias en operaciones de escritura.

5. Pruebas Unitarias Automatizadas (Ítem e)

La suite implementada en `tests.py` cumple con los requisitos de la **Sección 9**: 10 pruebas unitarias sobre más de 5 funciones del sistema, sin emplear clases y con compatibilidad dual:

#	Categoría	Función Testeada	Datos y Situación Evaluada	Resultado
01	Válido	`hotel.inicializar_hotel`	pisos = 2, habitaciones = 3 (dimensiones válidas)	APROBADO
02	Válido	`tarifas.obtener_tarifa_habitacion`	piso = 2 en matriz con 3 pisos configurados	APROBADO
03	Válido	`reportes.calcular_subtotal`	estadía de 5 noches (≤ 7) a tarifa plena ($50.000)	APROBADO
04	Válido	`reportes.calcular_subtotal`	estadía de 10 noches (> 7) con 10% descuento ($90.000)	APROBADO
05	Inválido	`deteccion_dni_duplicado_set`	intento de registrar DNI '11111111' ya existente en el set	APROBADO
06	Inválido	`hotel.habitacion_existe`	consulta de coordenadas piso = 5 en matriz de 2x2	APROBADO
07	Inválido	`huespedes.buscar_huesped...`	búsqueda en habitación 999 sin huéspedes (retorna [])	APROBADO
08	Límite	`reportes.calcular_subtotal`	exactamente 7 noches (frontera estricta sin descuento)	APROBADO
09	Límite	`hotel.inicializar_hotel`	hotel en dimensión mínima permitida (1 piso, 1 hab)	APROBADO
10	Adicional	`huespedes.buscar_por_nombre`	búsqueda parcial con `filter` y `lambda` ("juan")	APROBADO

Formas de Ejecución:
- Con Python nativo: `python tests.py` (emite informe completo con los 5 puntos exigidos)
- Con PyTest: `python -m pytest tests.py -v` (10 passed in 0.10s)

6. Cuadro Comparativo: Fase 1 vs. Fase 2 (Ítem f)

Criterio Evaluado	Primera Entrega (Fase 1)	Segunda Entrega (Fase 2)
Estructura del Proyecto	Código centralizado monolítico	Arquitectura modular desacoplada en 7 módulos especializados
Modelado de Huéspedes	Sublistas indexadas por posición `[nom, dni, ...]`	Diccionarios asociativos `dict` con tuplas inmutables `tuple`
Control de Unicidad	Recorrido lineal O(n) por bucle	Búsqueda en tiempo constante O(1) mediante conjuntos `set`
Cálculo de Disponibilidad	Bucles anidados sobre la matriz	Álgebra de conjuntos (operación diferencia `todas - ocupadas`)
Persistencia de Datos	Inexistente (datos volátiles en memoria RAM)	Triple persistencia: JSON (estado), TXT (bitácora) y CSV (Excel)
Control de Errores	Validaciones básicas condicionales	Manejo estructurado de excepciones con `try / except / else / finally`
Calidad de Software	Pruebas manuales por consola	Suite automatizada de 10 pruebas unitarias con aserciones nativas
Restricciones de Cátedra	Sin clases ni librerías no vistas	100% libre de clases, bases de datos o librerías externas no autorizadas

7. Conclusión y Respaldo Académico

El sistema **Room Master** cumple de manera estricta y verificable con el 100% de las directivas impartidas por la cátedra del **Lic. Gustavo E. Escandell** para la materia **Programación 1 / Algoritmo y Estructura de Datos 1 (UADE 2026)**.

La solución final demuestra que es posible concebir una arquitectura de software robusta, testeada y mantenible explotando el poder de la **programación estructurada, modular y funcional** nativa de Python, sin necesidad de recurrir a la Programación Orientada a Objetos ni a dependencias externas.